

Week03_Gate_03 Assessment Document

Project Overview

This project automates the end-to-end retail shopping journey using Selenium WebDriver, Cucumber BDD, and Java.

The framework validates the complete customer purchase flow from searching a product to confirming an order. It follows the Page Object Model (POM) design pattern to improve code reusability, maintainability, and readability.

Objective

The objective of this project is:

1. Open Catalog
2. Search Product
3. Add Product to Cart
4. Open Cart
5. Verify Cart Details
6. Proceed to Checkout
7. Place Order
8. Verify Order Confirmation

The framework also includes a negative test case to demonstrate failure handling and screenshot capture.

Technologies Used

- Java
- Selenium WebDriver
- Cucumber BDD
- JUnit 5
- Maven
- Chrome Browser

Framework Design

The framework follows the Page Object Model (POM).

Pages

CatalogPage

- Opening catalog page
- Searching products
- Sorting products
- Opening product details

ProductPage

- Adding product to cart

CartPage

- Viewing cart items
- Verifying line item count
- Proceeding to checkout

CheckOutPage

- Placing order
- Reading confirmation message

Components

Header

- Opening cart
- Accessing cart badge

CartBadge

- Verifying cart item count

ProductCard

- Reading product title
- Reading product price

Support Classes

Config

Reads configuration values from:

env.properties

Example:

- Base URL
- Browser settings
- Timeout values
- Window dimensions

DriverFactory

Creates and manages Chrome WebDriver.

World

Stores shared objects between step definitions and hooks.

Hooks

Handles:

- Browser launch before execution
- Browser close after execution
- Screenshot capture on failure

Test Scenarios

Smoke Test

Buy a single product end to end.

Flow:

Open Catalog → Search Product → Add To Cart → Open Cart → Checkout → Order Confirmed

Cart Badge Validation

Verify cart badge count after adding an item.

Expected:1

Cart badge displays "1".

Regression Test

Verify checkout flow for multiple products.

Products tested:

- Running Shoes
 - Travel Backpack
 - Noise Canceling Headphones
-

Empty Cart Validation

Verify a newly opened cart contains zero items.

Expected:

0 line items.

Negative Test

Add one product and intentionally verify cart badge count as "2".

Expected Result:

Test fails.

Purpose:

- Demonstrate failure handling
 - Demonstrate screenshot capture functionality
 - Validate reporting for failed scenarios
-

Results

Total Scenarios Executed: 7

Passed: 6

Failed: 1

The failed scenario is intentional and used to verify screenshot capture functionality.

```
Run RunCucumberTest x
1 test failed, 6 passed 7 tests total, 49 sec 563 ms
Retail catalog to checkout journey 49 sec 563 ms
  Buy a single product end to end 9 sec 89 ms
  Cart badge reflects the number of items added 6 sec 302 ms
  Buy several products through the reusable journey 21 sec 983 ms
    Examples
      Example #1.1 7 sec 120 ms
      Example #1.2 7 sec 244 ms
      Example #1.3 7 sec 619 ms
    A fresh cart has no line items 5 sec 754 ms
    Deliberate failure demonstration 6 sec 435 ms

@smoke
Scenario: Buy a single product end to end # classpath:features/checkout.feature
  Given the shopper opens the catalog # org.cucumber.sdet.bdd.CheckoutStep
  When the shopper searches for "Running Shoes" # org.cucumber.sdet.bdd.CheckoutStep
  And adds the product to the cart # org.cucumber.sdet.bdd.CheckoutStep
  And the shopper opens the cart # org.cucumber.sdet.bdd.CheckoutStep
  Then the cart contains 1 line item # org.cucumber.sdet.bdd.CheckoutStep
  When the shopper captures the cart total and completes checkout # org.cucumber.sdet.bdd.CheckoutStep
  Then the order is confirmed # org.cucumber.sdet.bdd.CheckoutStep
```

```
Run RunCucumberTest x
1 test failed, 6 passed 7 tests total, 49 sec 563 ms
Retail catalog to checkout journey 49 sec 563 ms
  Buy a single product end to end 9 sec 89 ms
  Cart badge reflects the number of items added 6 sec 302 ms
  Buy several products through the reusable journey 21 sec 983 ms
    Examples
      Example #1.1 7 sec 120 ms
      Example #1.2 7 sec 244 ms
      Example #1.3 7 sec 619 ms
    A fresh cart has no line items 5 sec 754 ms
    Deliberate failure demonstration 6 sec 435 ms

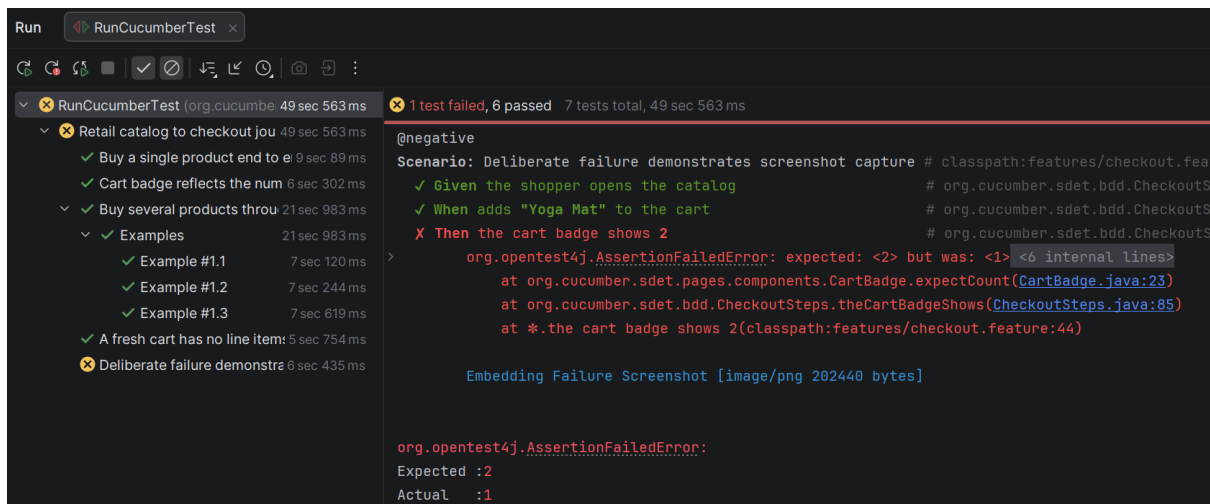
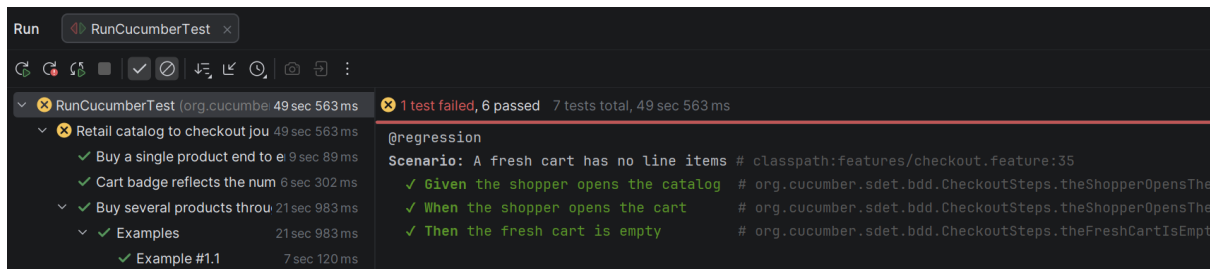
@smoke
Scenario: Cart badge reflects the number of items added # classpath:features/checkout.feature
  Given the shopper opens the catalog # org.cucumber.sdet.bdd.CheckoutStep
  When adds "Travel Backpack" to the cart # org.cucumber.sdet.bdd.CheckoutStep
  Then the cart badge shows 1 # org.cucumber.sdet.bdd.CheckoutStep

@regression
Scenario Outline: Buy several products through the reusable journey # classpath:features/checkout.feature
  Given the shopper opens the catalog # org.cucumber.sdet.bdd.CheckoutStep
  When adds "Running Shoes" to the cart # org.cucumber.sdet.bdd.CheckoutStep
  And the shopper opens the cart # org.cucumber.sdet.bdd.CheckoutStep
  Then the cart contains 1 line item # org.cucumber.sdet.bdd.CheckoutStep
  When the shopper captures the cart total and completes checkout # org.cucumber.sdet.bdd.CheckoutStep
  Then the order is confirmed # org.cucumber.sdet.bdd.CheckoutStep
```

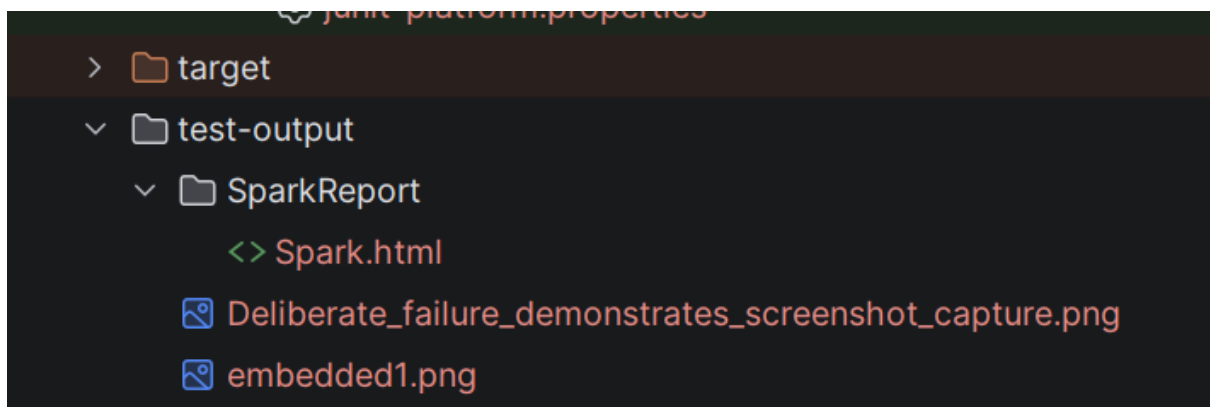
```
Run RunCucumberTest x
1 test failed, 6 passed 7 tests total, 49 sec 563 ms
Retail catalog to checkout journey 49 sec 563 ms
  Buy a single product end to end 9 sec 89 ms
  Cart badge reflects the number of items added 6 sec 302 ms
  Buy several products through the reusable journey 21 sec 983 ms
    Examples
      Example #1.1 7 sec 120 ms
      Example #1.2 7 sec 244 ms
      Example #1.3 7 sec 619 ms
    A fresh cart has no line items 5 sec 754 ms
    Deliberate failure demonstration 6 sec 435 ms

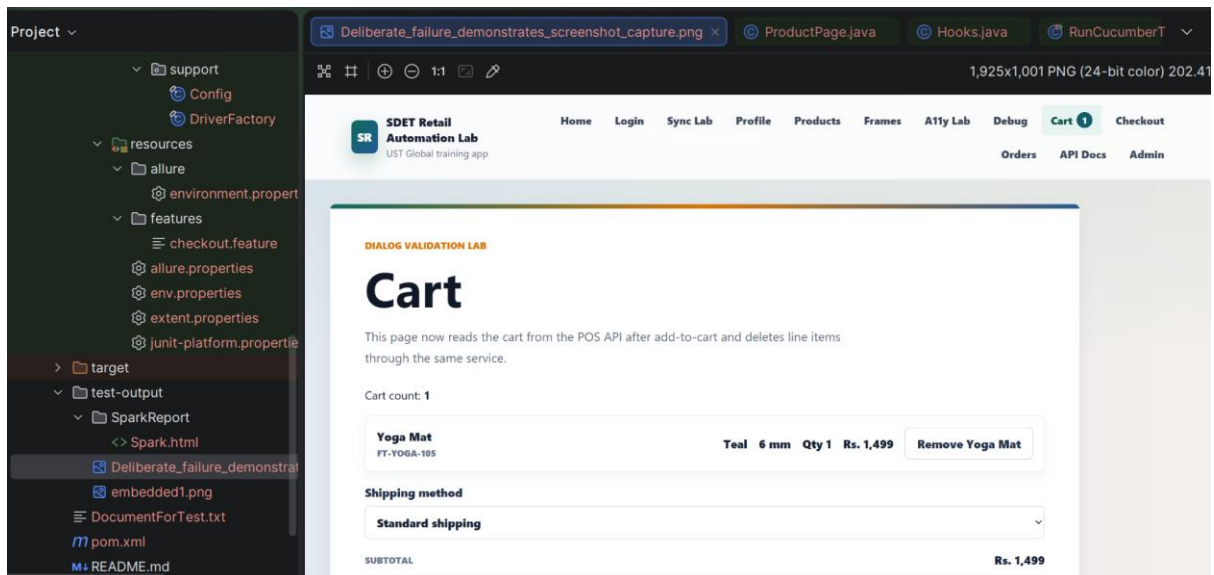
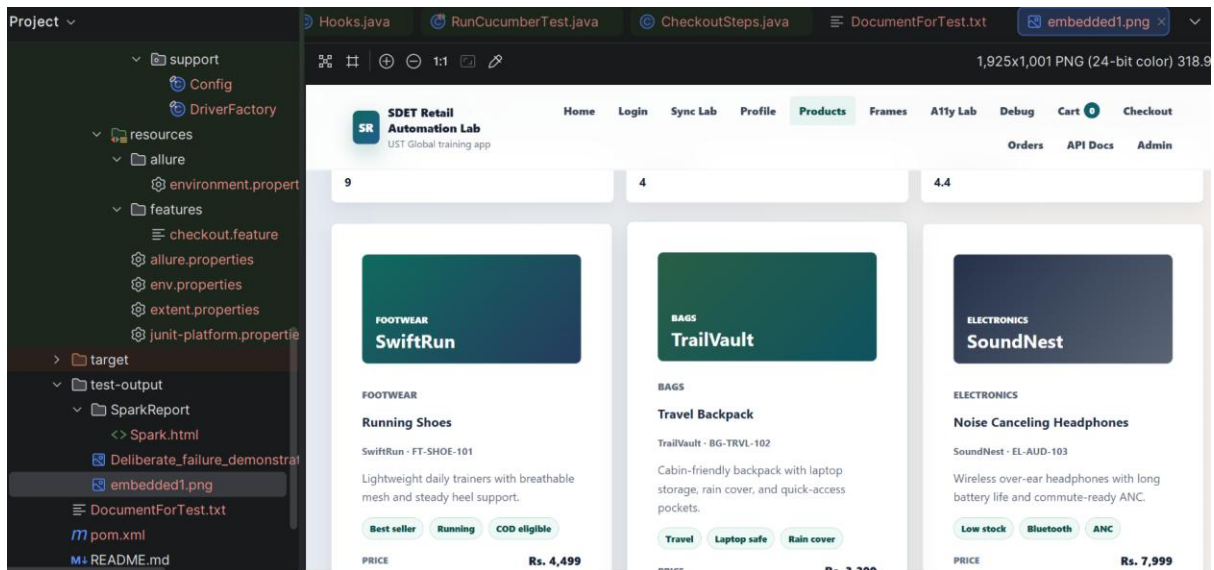
@regression
Scenario Outline: Buy several products through the reusable journey # classpath:features/checkout.feature
  Given the shopper opens the catalog # org.cucumber.sdet.bdd.CheckoutStep
  When adds "Travel Backpack" to the cart # org.cucumber.sdet.bdd.CheckoutStep
  And the shopper opens the cart # org.cucumber.sdet.bdd.CheckoutStep
  Then the cart contains 1 line item # org.cucumber.sdet.bdd.CheckoutStep
  When the shopper captures the cart total and completes checkout # org.cucumber.sdet.bdd.CheckoutStep
  Then the order is confirmed # org.cucumber.sdet.bdd.CheckoutStep

@regression
Scenario Outline: Buy several products through the reusable journey # classpath:features/checkout.feature
  Given the shopper opens the catalog # org.cucumber.sdet.bdd.CheckoutStep
  When adds "Noise Canceling Headphones" to the cart # org.cucumber.sdet.bdd.CheckoutStep
  And the shopper opens the cart # org.cucumber.sdet.bdd.CheckoutStep
  Then the cart contains 1 line item # org.cucumber.sdet.bdd.CheckoutStep
  When the shopper captures the cart total and completes checkout # org.cucumber.sdet.bdd.CheckoutStep
  Then the order is confirmed # org.cucumber.sdet.bdd.CheckoutStep
```

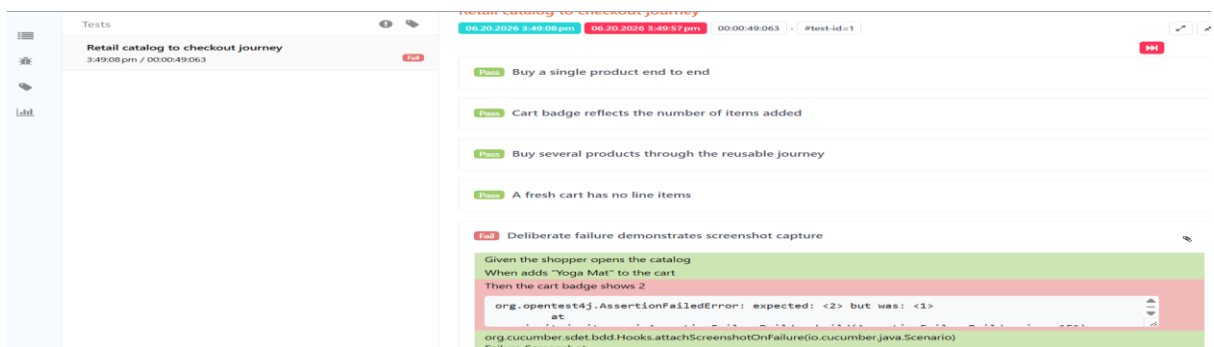


Test-Output





Spark.html



Cucumber.html

cucumber-jvm@7.34.3 with OpenJDK 64-Bit Server VM@22.0.2+9-adhoc.Administrator.jdk22u on Windows 11@10.0

85.7% (6 / 7) passed 46 minutes ago in 49 seconds

Search with text or @tags PASSED 6 FAILED 1

Scenarios

classpath:features/checkout.feature

Feature: Retail catalog to checkout journey

@smoke

Scenario: Buy a single product end to end

- Given the shopper opens the catalog 784ms
- When the shopper searches for "Running Shoes" 386ms
- And adds the product to the cart 301ms
- And the shopper opens the cart 133ms
- Then the cart contains 1 line item 23ms
- When the shopper captures the cart total and completes checkout 213ms
- Then the order is confirmed 558ms

3 hooks

@negative

Scenario: Deliberate failure demonstrates screenshot capture

- Given the shopper opens the catalog 922ms
- When adds "Yoga Mat" to the cart 590ms
- Then the cart badge shows 2 74ms

```
org.opentest4j.AssertionFailedError: expected: <2> but was: <1>
org.opentest4j.AssertionFailedError: expected: <2> but was: <1>
    at org.junit.jupiter.api.AssertionFailureBuilder.build(AssertionFailureBuilder.java:151)
    at org.junit.jupiter.api.AssertionFailureBuilder.buildAndThrow(AssertionFailureBuilder.java:132)
    at org.junit.jupiter.api.AssertEquals.failNotEqual(AssertEquals.java:197)
    at org.junit.jupiter.api.AssertEquals.assertEquals(AssertEquals.java:182)
    at org.junit.jupiter.api.AssertEquals.assertEquals(AssertEquals.java:177)
    at org.junit.jupiter.api.Assertions.assertEquals(Assertions.java:1145)
    at org.cucumber.sdet.pages.components.CartBadge.expectCount(CartBadge.java:23)
    at org.cucumber.sdet.bdd.CheckoutSteps.theCartBadgeShows(CheckoutSteps.java:85)
    at *.the cart badge shows 2(classpath:features/checkout.feature:44)
```

3 hooks

Maven Execution Note

Direct execution using Maven commands could not be completed in the current environment because of SSL certificate validation issues .

Error:

PKIX path building failed

The automation tests were executed successfully through IntelliJ IDEA where all required dependencies were already available locally.

Commands and Expected Output:

1. mvn test -Dcucumber.filter.tags='@negative'

Executes only smoke test scenarios to validate the critical end-to-end checkout flow.

2. mvn test -Dcucumber.filter.tags='@regression'

Executes all regression scenarios to validate core application functionality across multiple test cases.

3. `mvn test -Dcucumber.filter.tags='@smoke or @regression'`

Executes both smoke and regression scenarios to provide broader test coverage of the application.

4. `mvn test -Dcucumber.filter.tags='@negative'`

Executes only negative test scenarios and verifies application behavior for invalid or failure conditions, including screenshot capture on failure.

Key Features

- Page Object Model implementation
- Reusable components
- Centralized configuration
- Screenshot capture on failure
- Cucumber BDD reporting
- End-to-end checkout automation

Conclusion

This project successfully automates the retail catalog-to-checkout journey using Selenium and Cucumber. The framework is scalable, reusable, and easy to maintain. It validates both positive and negative scenarios while demonstrating proper test automation practices.